[CE310] Evolutionary Computation and Genetic Programming

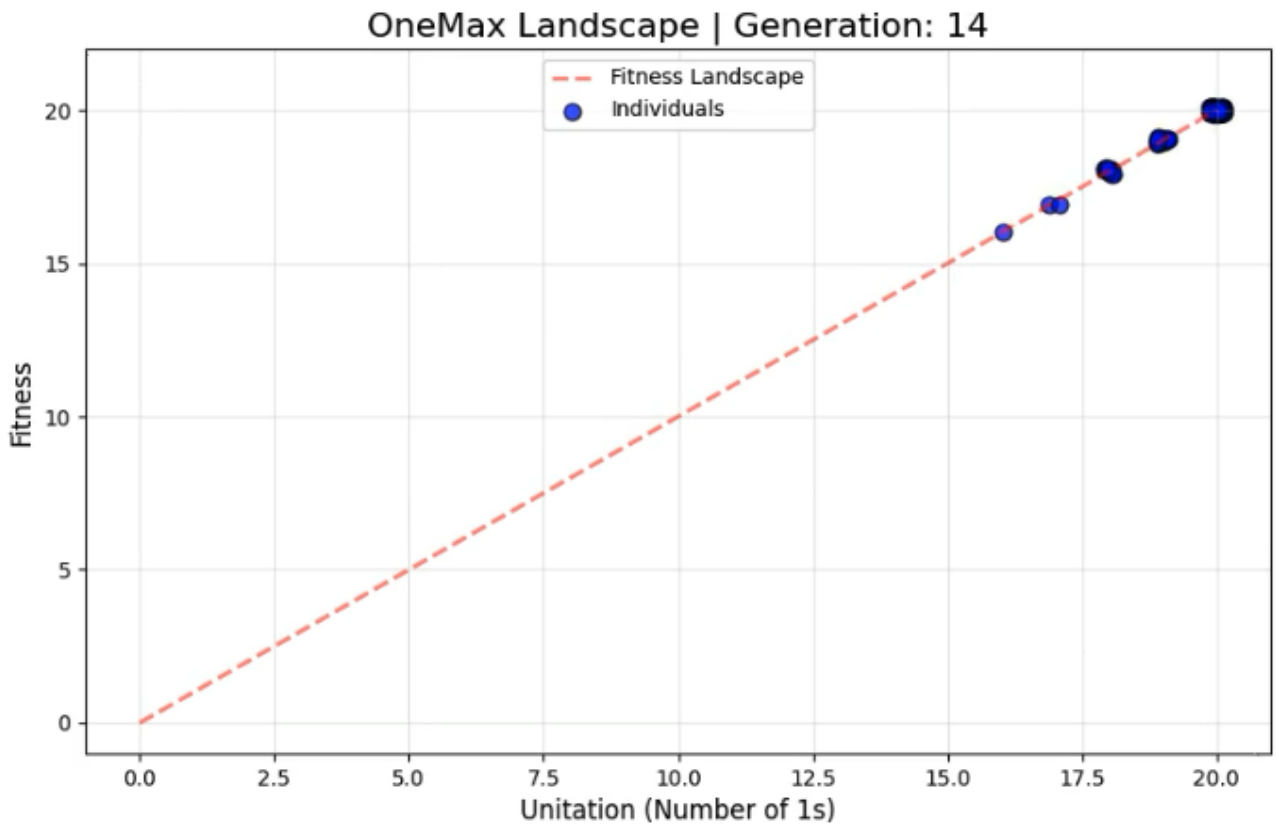
Riccardo Poli

Evolution strategies

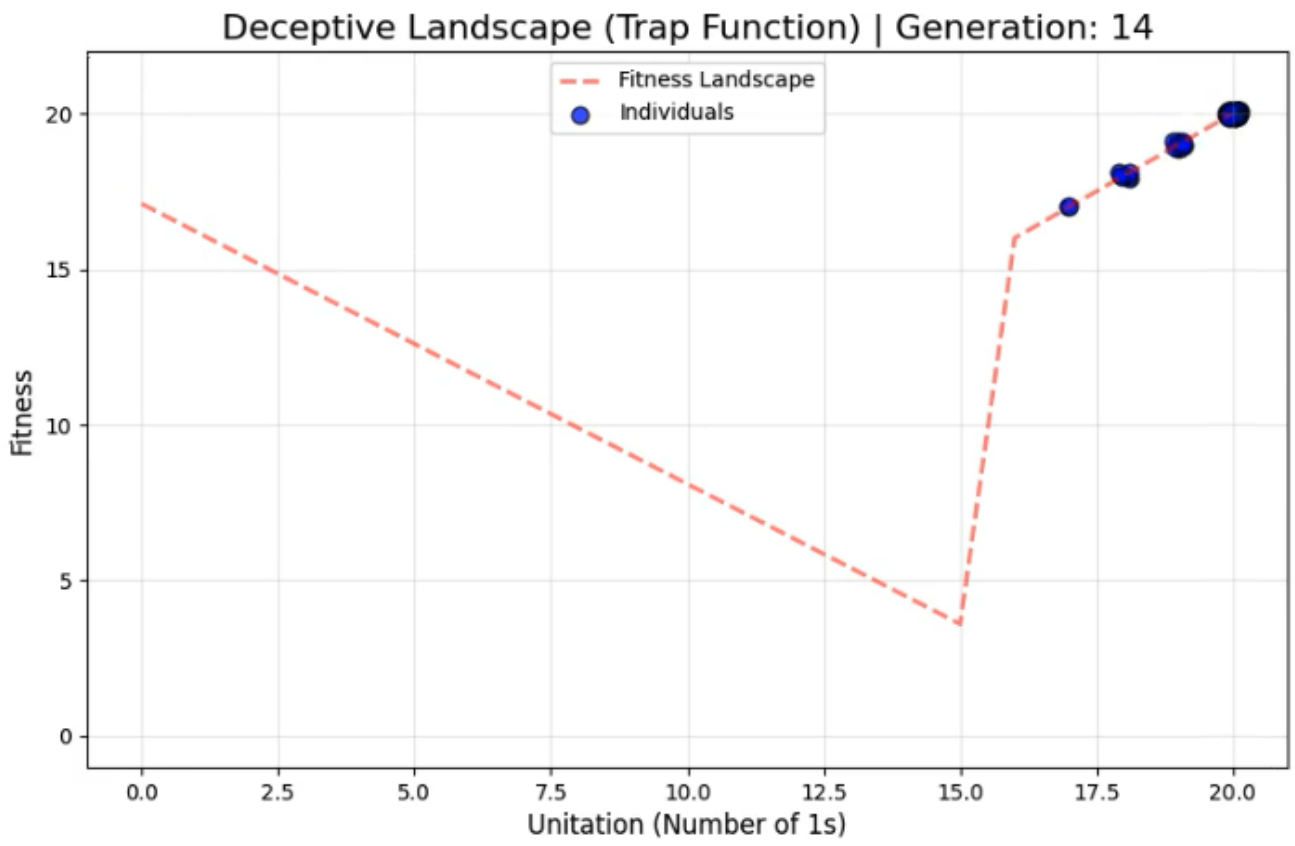
University of Essex
School of Computer Science and Electronic Engineering

Post Lab Addendum

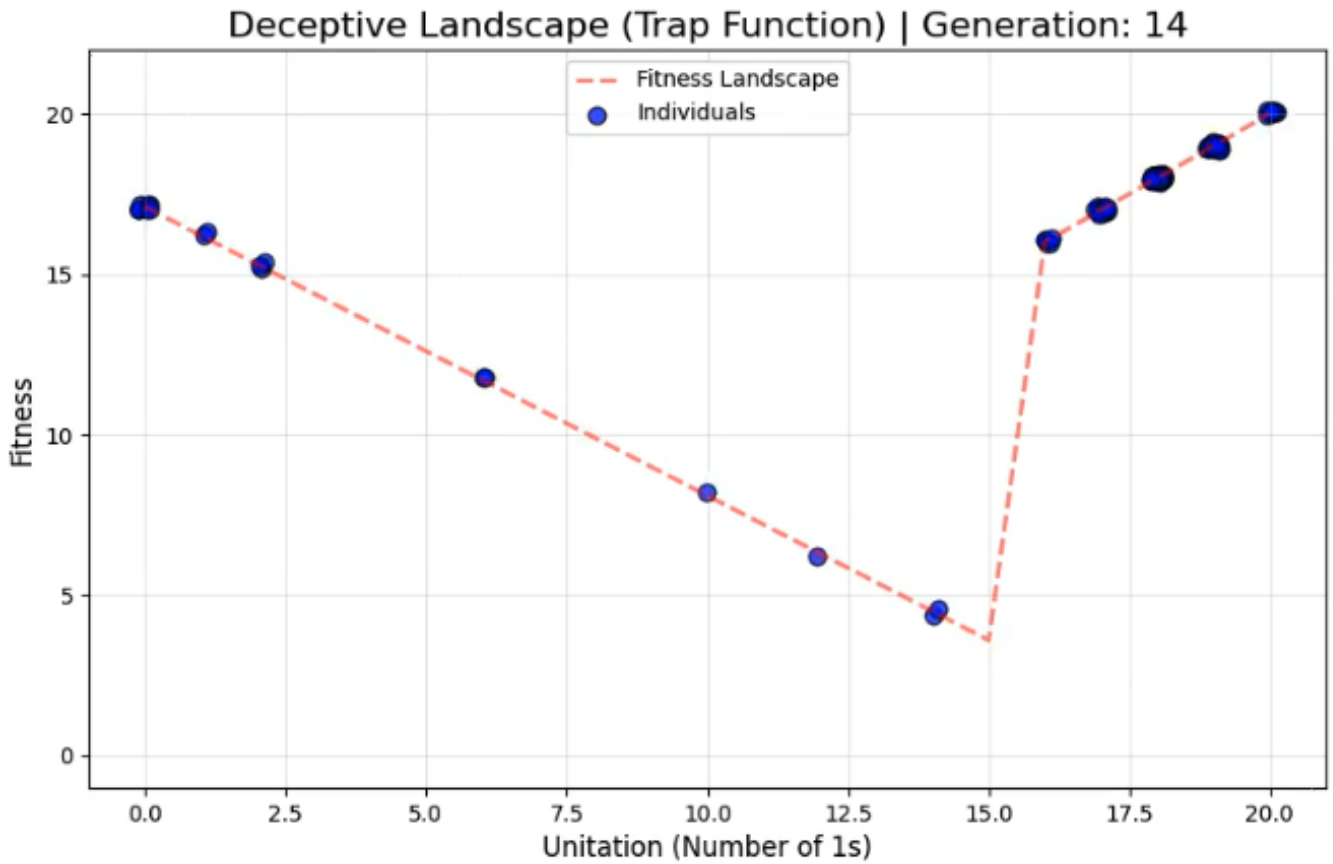
Binary Steady State GA on OneMax function



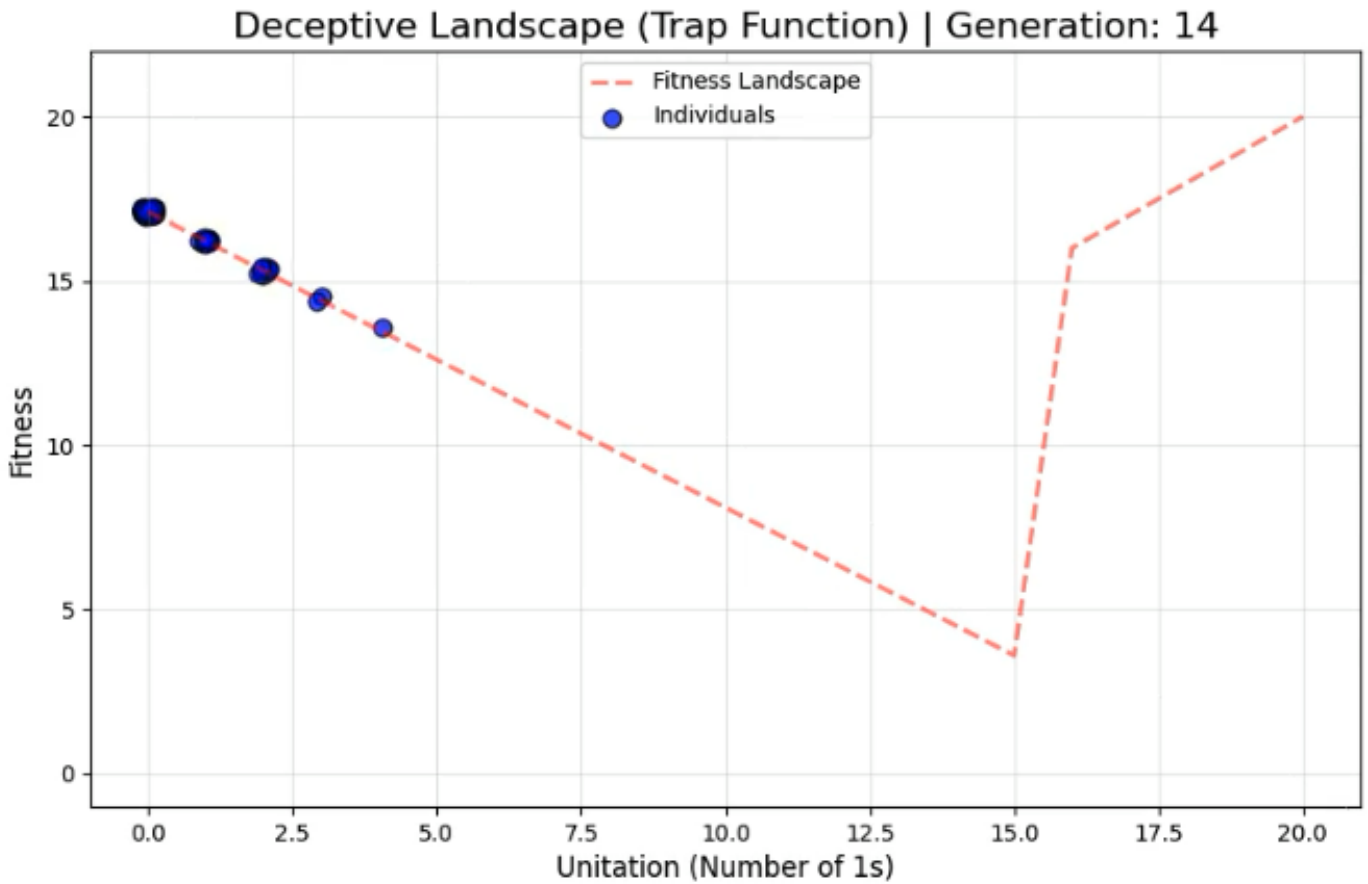
Binary Steady State GA on deceptive/trap function



Binary Steady State GA on deceptive/trap function



Binary Steady State GA on deceptive/trap function



Today’s learning objective

- Describe **evolution strategies** (ESs) algorithm and possible ways to implement them.
- Discuss and contrast **evolution strategies** (EAs) and **genetic algorithms** (GAs)

Evolution Strategies

- An **Evolution Strategy** (ES) is EAs which uses arrays of real numbers as a representation, like real-valued GAs.
- **Mutation** is the key variation operator (unlike GAs, where recombination is usually deemed to play that role)
- **Mutation** = **addition of Gaussian random numbers** to the floats that represent the individuals being optimised.
- The **amplitude of deviates is varied dynamically** to maximise performance (but often ESs are also used with fixed mutation).

Gaussian Mutation

- Same implementation as mutation in real-valued GAs:

$$\text{Offspring}_{Param1} = \text{Parent}_{Param1} + \text{sigma} \cdot R_1$$

$$\ldots$$

$$\text{Offspring}_{ParamN} = \text{Parent}_{ParamN} + \text{sigma} \cdot R_N$$

but

$R_1, \ldots, R_N$ = random numbers drawn from a **standardised Gaussian distribution**

Gaussian Mutation

Same implementation as mutation in real-valued GAs but the R_i are NOT uniformly distributed

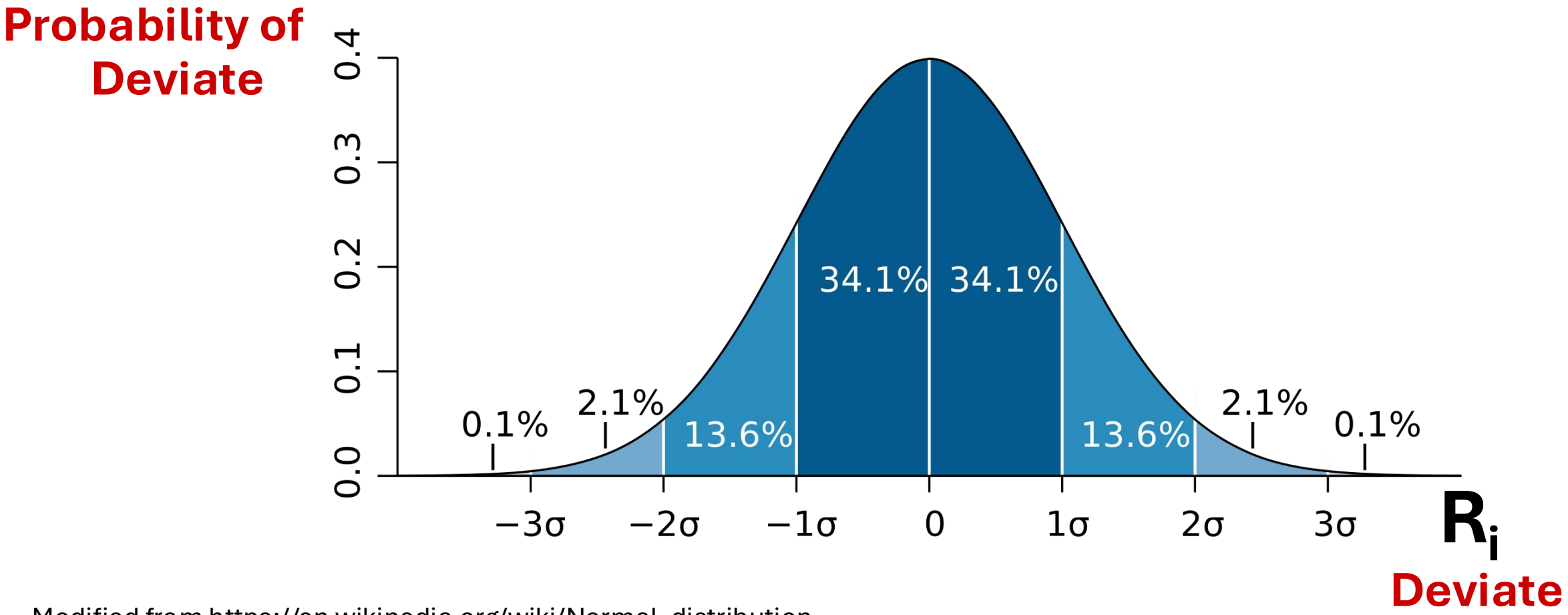
$$\begin{aligned} \text{Offspring}[1] &= \text{Parent}[1] + \text{sigma} \cdot \mathrm{R}_1 \\ &\vdots \\ \text{Offspring}[N] &= \text{Parent}[\mathrm{N}] + \text{sigma} \cdot \mathrm{R}_N \end{aligned}$$

where:

- *sigma* is a suitable positive constant and
- $\mathrm{R}_1, \ldots, \mathrm{R}_\mathrm{N}$ are (different) random numbers drawn from a <u>normal/gaussian</u> distribution
- In python:

```
import random
Ri = random.gauss(0, sigma)
```

Standardised Gaussian Deviates

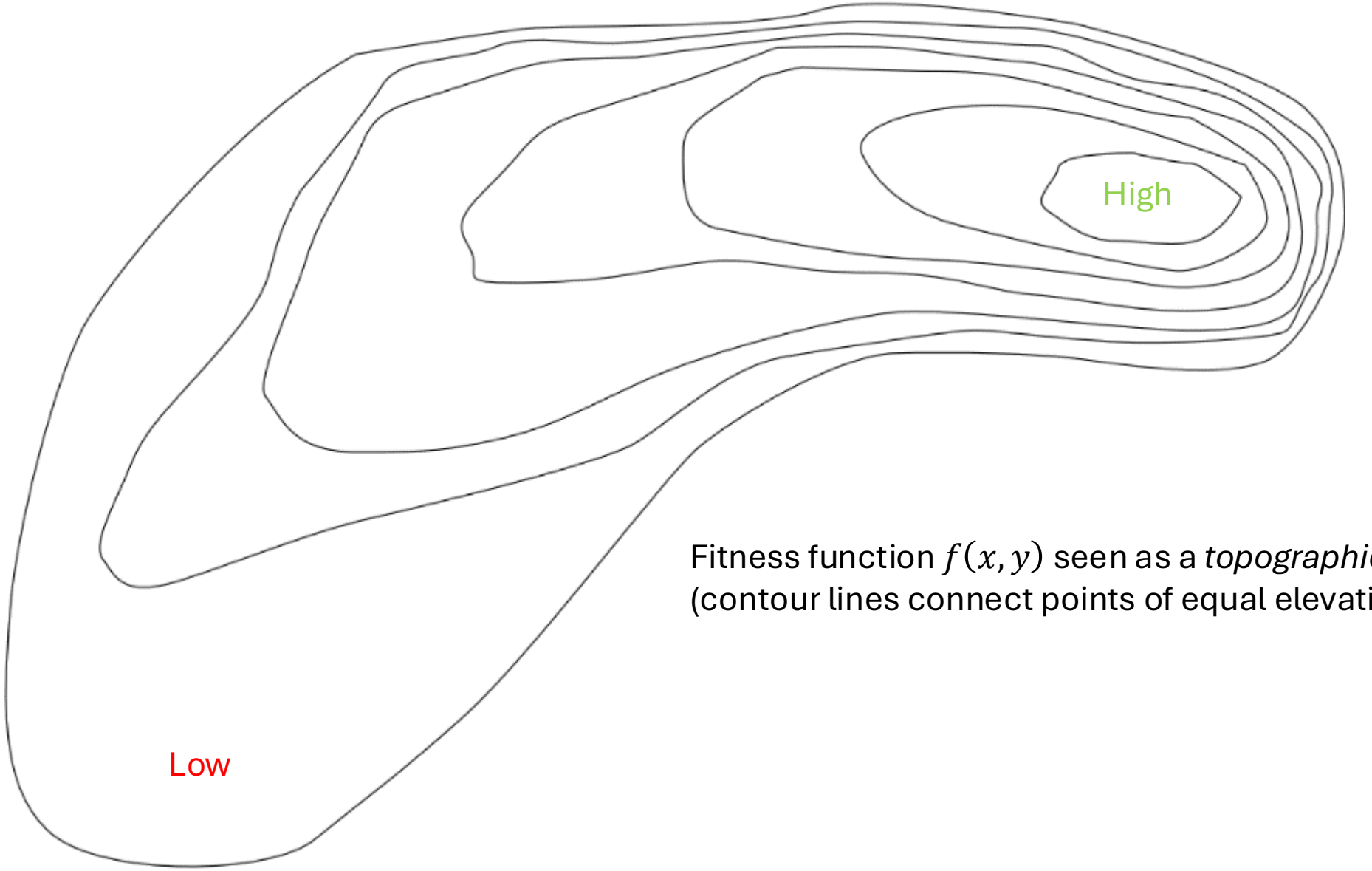


Modified from https://en.wikipedia.org/wiki/Normal_distribution

(1+1) ES

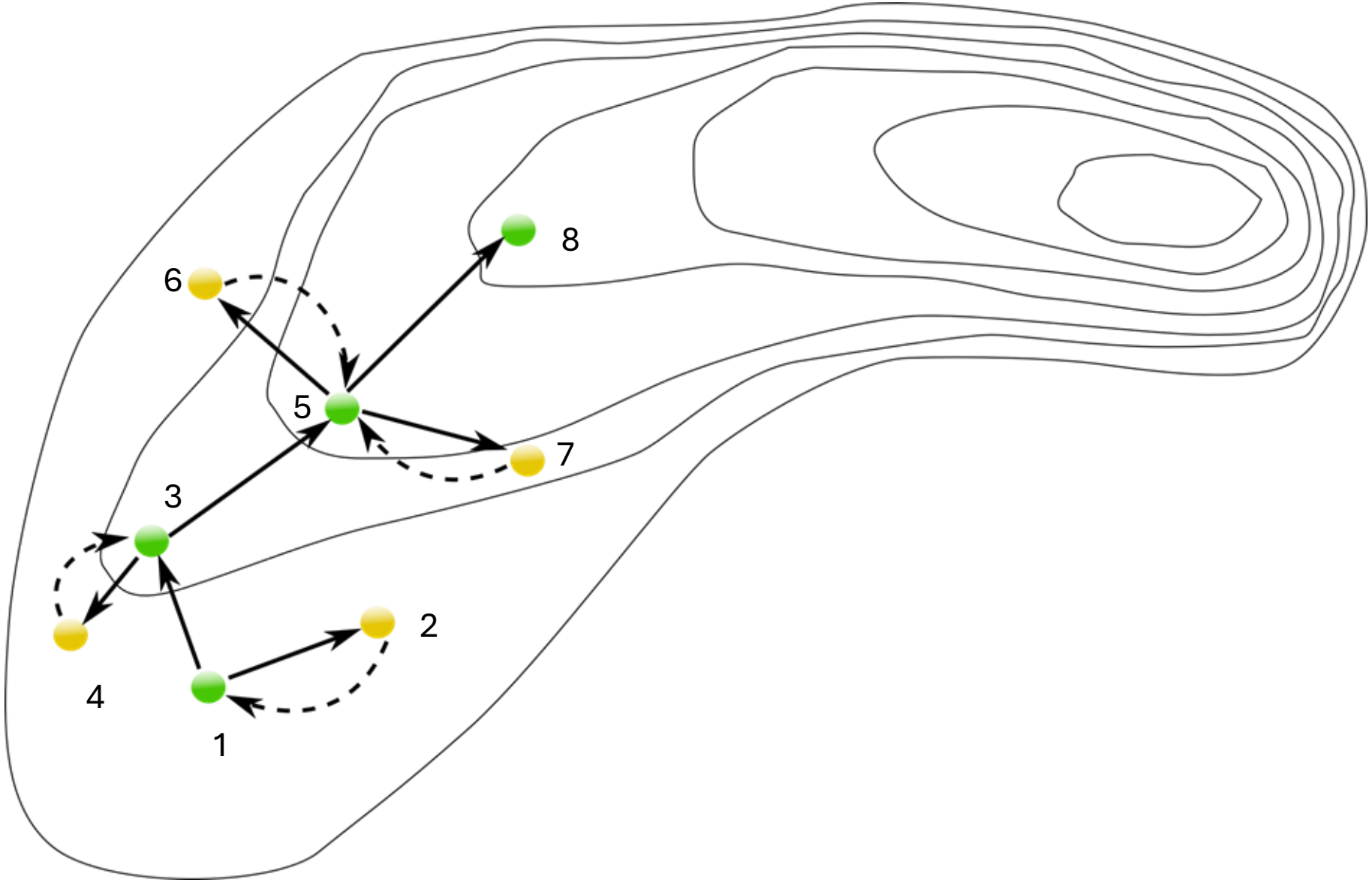
- The (**1+1**) **ES** uses a population consisting of only one individual (the first “1” in “(**1**+1)”).
- The parent generates one offspring (the second “1” in “(1+**1**)”) per generation by applying Gaussian mutations.
- If offspring performs no worse than its parent, it replaces it.

Sample 2-D Fitness Landscape



Fitness function $f(x, y)$ seen as a *topographic map* (contour lines connect points of equal elevation)

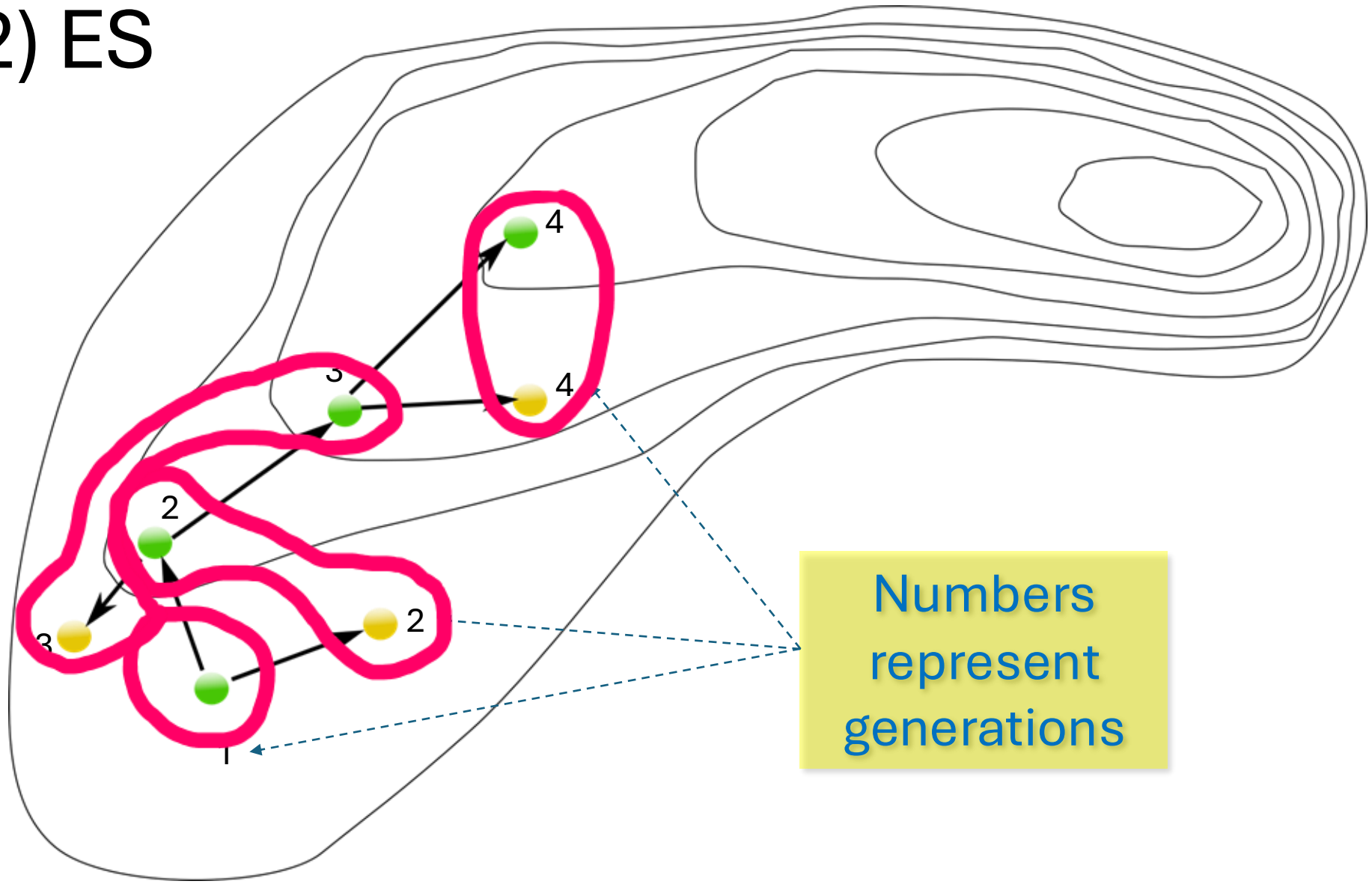
(1+1)-ES Behaviour



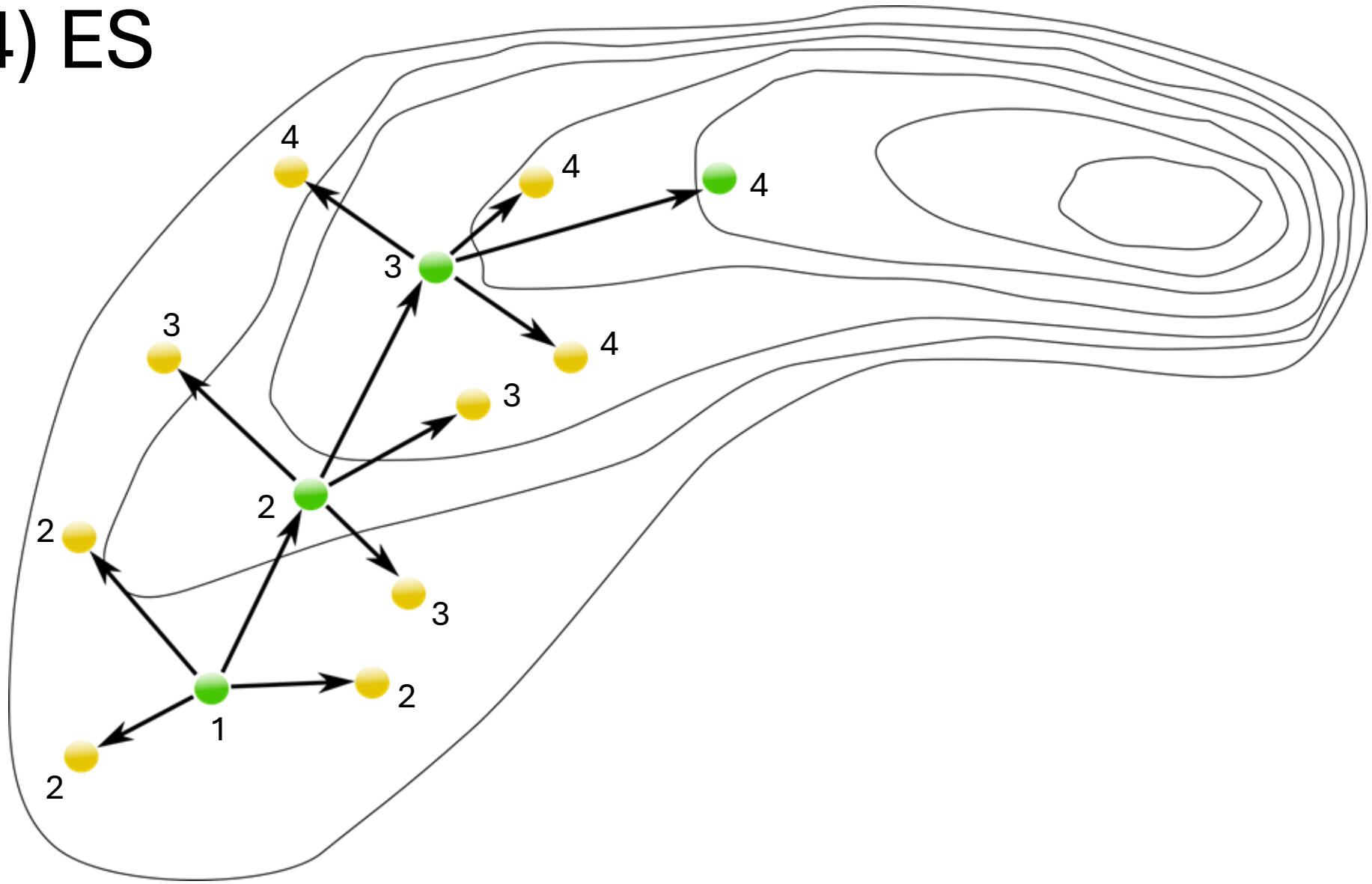
(1+lambda), (mu+lambda) and (mu,lambda) ES

- **(1+lambda**) ES is like the (1+1) ES except it generates **lambda offspring** and it chooses the best as a candidate for replacing the parent.
- **(mu+lambda)** ES produces **lambda offspring** at each generation **via crossover and mutation**. The best mu individuals out of the lambda + mu available ones are retained.
- **(mu, lambda)** ES is like (mu+lambda) ES but it the **replaces the parents with the best mu offspring out of lambda generated** (naturally, lambda > mu for this to work).

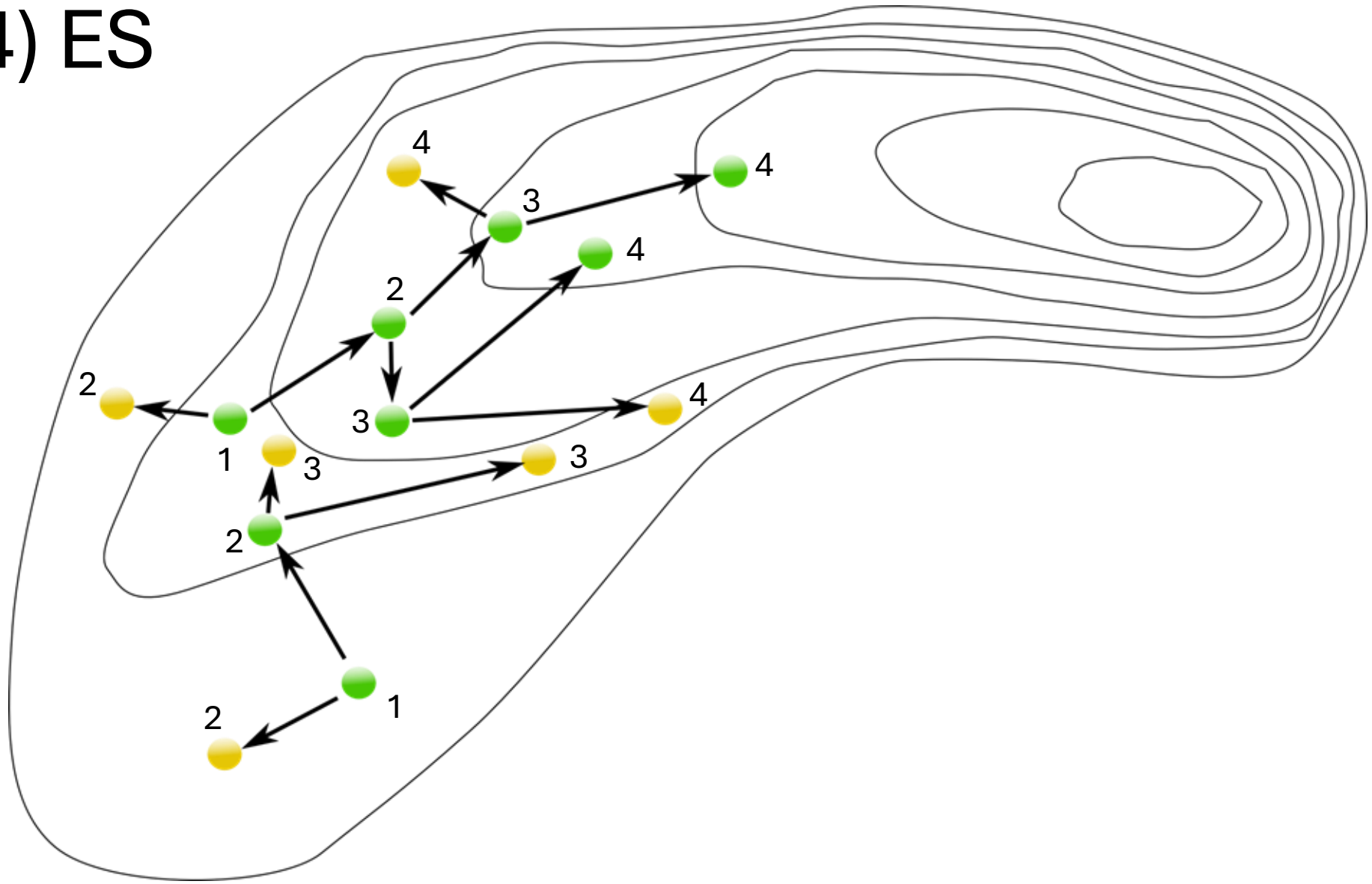
(1,2) ES



(1,4) ES



(2,4) ES



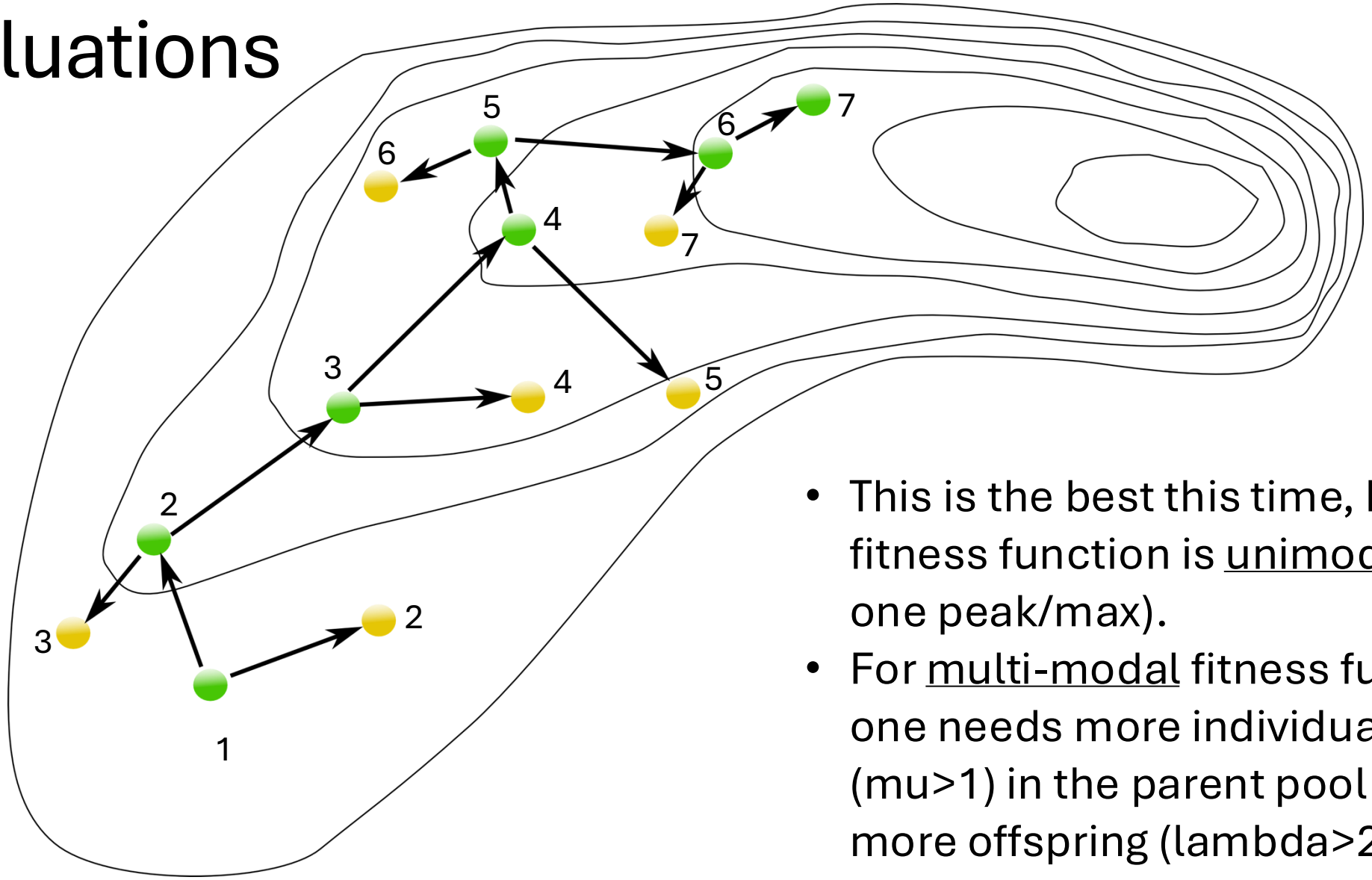
Which ES is best?

- It might seem that (1,4) and (2,4) ES are **best**.
- However, more offspring means more fitness evaluations, i.e., longer runtimes

Our example ES	Fitness evaluations
1+1	8
1,2	7
1,4	13
2,4	14

- So, fair comparison should be done on approximately same number of fitness evaluations

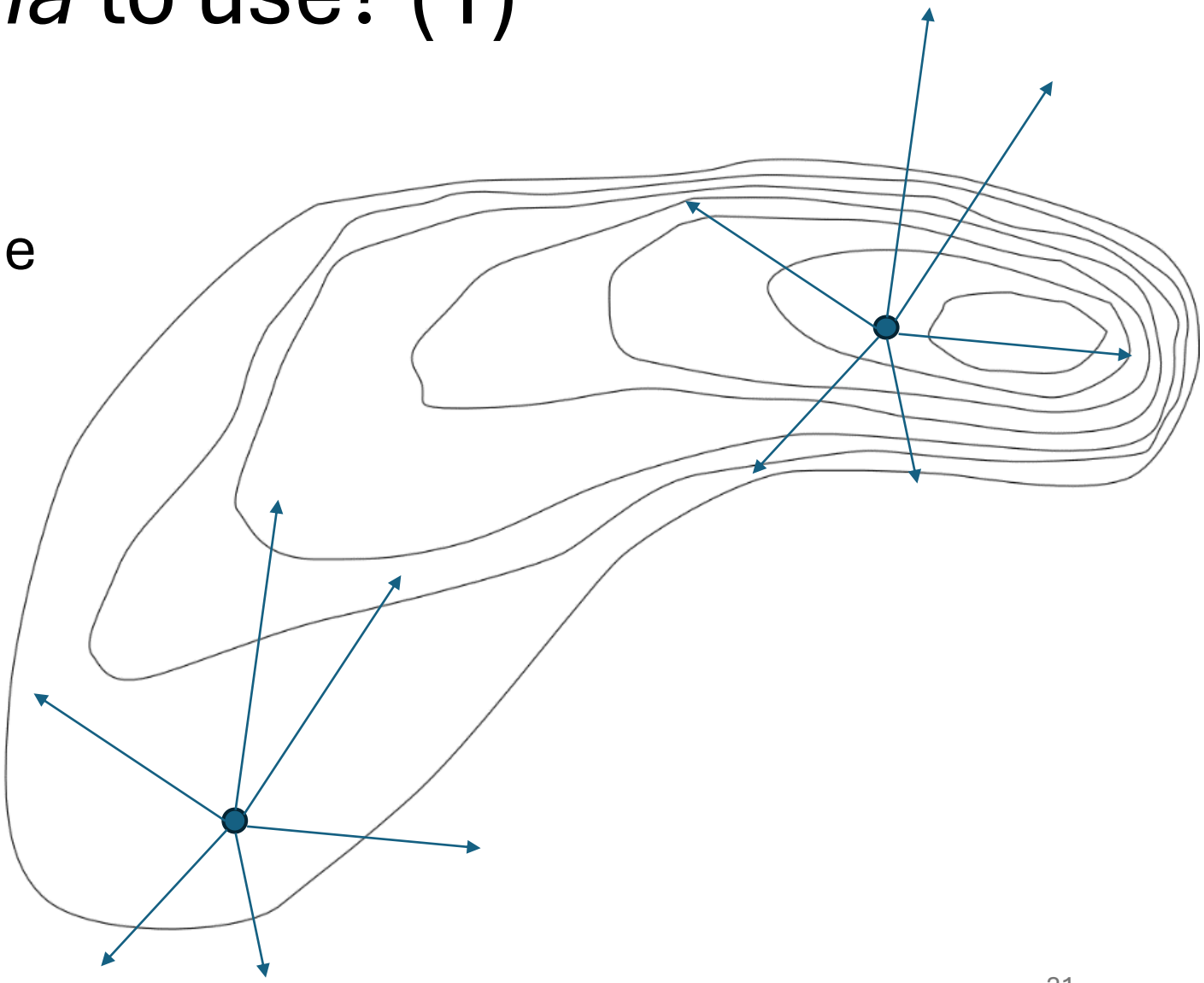
(1,2) ES run extended to 13 fitness evaluations



- This is the best this time, but the fitness function is <u>unimodal</u> (only one peak/max).
- For <u>multi-modal</u> fitness functions one needs more individuals (mu>1) in the parent pool and more offspring (lambda>2)

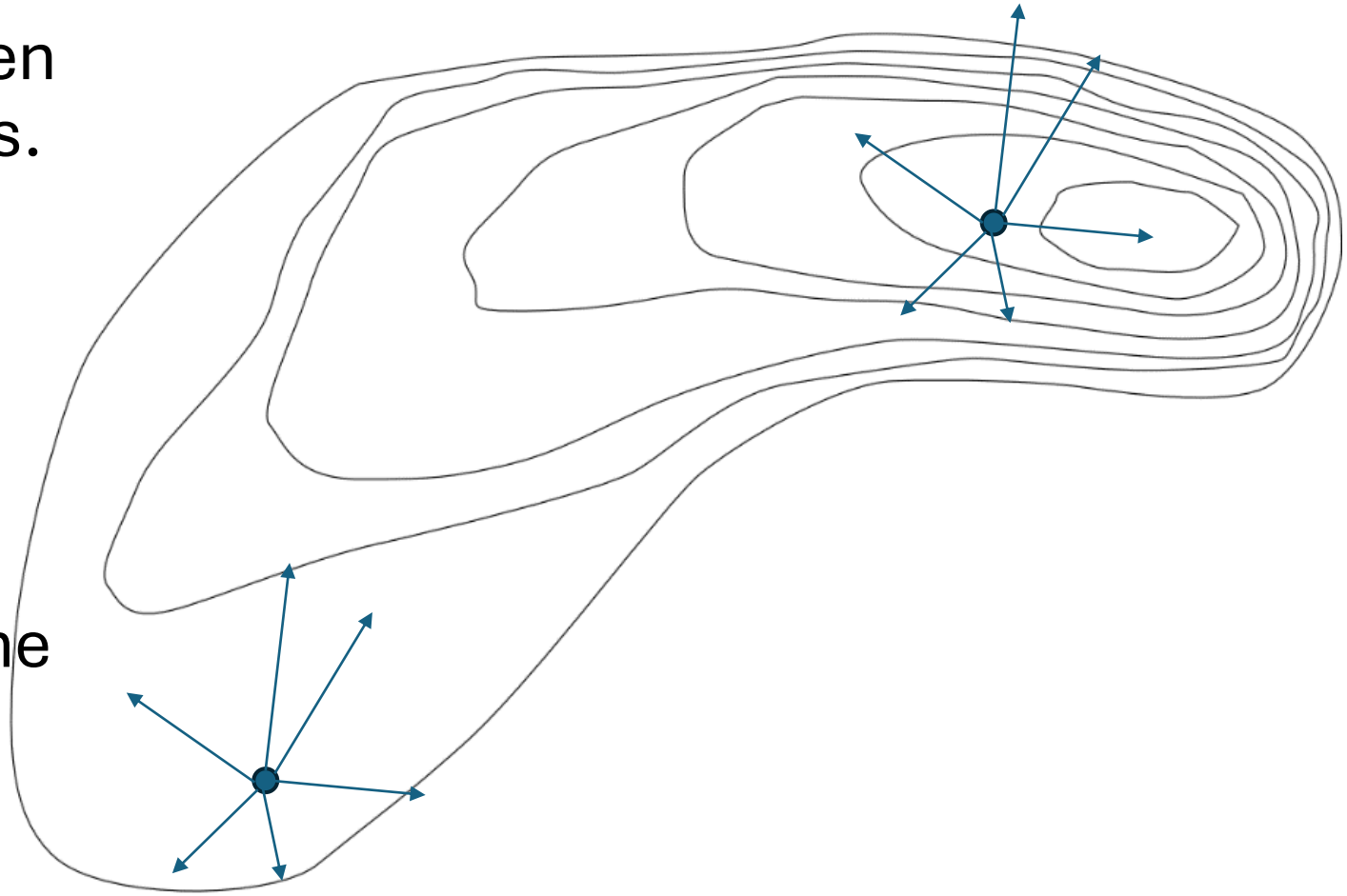
Which value of *sigma* to use? (1)

- **Big** *sigma* means children will be further away from the parents.
- Initially this may lead to bigger improvements in fitness
- However, the closer the population gets to the optimum the harder it gets to home on it.



Which value of *sigma* to use? (2)

- **Small** *sigma* means children will be closer to the parents.
- Initially this may lead to smaller improvements in fitness
- However, the closer the population gets to the optimum it is easier to home on it.



Sigma Adaptation and 1/5 Rule

- This issue is addressed by the **1/5 rule,** where **sigma** is changed according to the frequency of successful mutations
- We define

$$s_{mut} = \underline{\text{fraction of times children are fitter than parents}}$$

- If $s_{mut} > 1/5$ ➔ increase sigma

  # we're exploring too little, e.g., on a gentle slope
- If $s_{mut} < 1/5$ ➔ decrease sigma

  # we're exploring too widely, e.g., near the optimum
- If $s_{mut} = 1/5$ ➔ do nothing # we're exploring just right!

[CE310] Evolutionary Computation and Genetic Programming

Riccardo Poli

Next week:
Genetic Programming

University of Essex
School of
Computer Science and
Electronic Engineering